

**SIMATS ENGINEERING**

**ASSESSMENT TOOL 1**

**COURSE NAME: Artificial Intelligence**

**COURSE CODE: CSA17**

**COURSE OUTCOME COVERED**

***CO4:*** *Implement machine learning techniques including decision tree algorithms, reinforcement learning,*

*and build models for classification and decision-making. (BL3)*

*Assessment Tool Weightage: 50%*

**Duration: 1 Hour**

**QUESTIONS: 4**

**Total: Marks:40**

Annexure A

***Code of Conduct:***

*I \_\_\_\_ R. Akshaya (192524118)\_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered*

*to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:R. Akshaya***

**Student 3: Akshaya R | Register Number: 192524118**

## **INDUSTRY PROBLEM TASK**

### **INDUSTRY SCENARIO**

**Context:** A retail chain has collected customer records (purchase frequency, average order value, complaint history, loyalty tenure, last purchase date) to build a predictive model for identifying customers likely to churn. The company also wants to train an AI agent to recommend retention actions (Discount / Loyalty Offer / Personal Outreach / No Action) based on a customer's engagement over time.

*You are hired as an AI/ML Engineer. Address the following tasks:*

#### **Task 1: Data Preparation & Inductive Learning**

- (a) Describe the inductive learning approach suitable for this retail dataset. Identify the target variable and at least three key features with justification.

**Inductive learning** learns a general pattern from previously observed customer records and uses that pattern to predict the outcome for new customers.

For this retail dataset, a **supervised classification approach** is suitable because historical customer data can be labelled according to whether the customer **churned or did not churn**.

- **Target variable:** Churn
  - 1 → Customer churned
  - 0 → Customer remained loyal

A suitable model could be **Decision Tree, Random Forest, Logistic Regression, or XGBoost**. The model learns the relationship between customer behaviour and churn from the training data.

#### **Key features:**

##### **1. Purchase Frequency**

- Shows how often a customer purchases.
- A decreasing or very low purchase frequency can indicate declining engagement and a higher probability of churn.

##### **2. Average Order Value (AOV)**

- Represents the average amount spent per purchase.
- Helps identify the customer's spending behaviour and overall value to the business.

##### **3. Complaint History**

- Indicates how frequently a customer has experienced or reported problems.

- A high number of complaints may indicate dissatisfaction and therefore a higher churn risk.

#### 4. Loyalty Tenure

- Measures how long the customer has been associated with the loyalty programme.
- Longer tenure can indicate stronger customer loyalty and potentially lower churn probability.

#### 5. Last Purchase Date

- Indicates how recently the customer interacted with the company.
- A long gap since the last purchase can be an important indicator of possible churn.

Therefore, the model learns from these historical patterns and predicts the churn probability of a new customer.

**(b)** Explain how you would handle class imbalance (far fewer churners than loyal customers) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).

#### Handling Class Imbalance

In this dataset, **churners are much fewer than loyal customers**, creating a class-imbalance problem.

For example:

- Loyal customers → 90%
- Churned customers → 10%

If the model is trained directly on this data, it may mostly predict "**loyal**" because that is the majority class. This can give high accuracy while performing poorly at identifying actual churners.

#### Suitable strategy: SMOTE

I would use **SMOTE (Synthetic Minority Over-sampling Technique)** to handle the imbalance.

SMOTE creates **synthetic examples of the minority class (churners)** based on existing churner records. This gives the model more examples from the minority class to learn from.

#### Procedure:

1. Split the dataset into training and testing sets.

2. Apply SMOTE **only to the training data**.
3. Train the model using the balanced training data.
4. Evaluate the model on the original, untouched test data.

Applying SMOTE only to training data is important because applying it before the split could cause **data leakage** and make the test results unreliable.

#### **Why SMOTE is suitable:**

The company is particularly interested in detecting customers who are likely to churn. SMOTE improves the model's ability to recognize the minority churn class without simply deleting useful loyal-customer records.

For evaluation, **precision, recall, F1-score and ROC-AUC** should be considered instead of relying only on accuracy.

- (c) Split the dataset (70:30) and justify the split ratio for a customer-retention use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

#### **(c) 70:30 Split and Preprocessing**

The dataset can be divided into:

- **70% → Training data**
- **30% → Testing data**

A **70:30 split** provides enough records for the model to learn customer behaviour while keeping a sufficiently large independent test set to evaluate how well the model performs on unseen customers.

For a churn-prediction system, this is useful because the company needs to know whether the model can correctly identify **new customers who may churn**, rather than only performing well on the customers used for training.

A **stratified split** should be used so that the proportion of churners and non-churners is approximately maintained in both sets.

#### **Preprocessing Steps**

##### **1. Handling missing values**

Missing values in features such as purchase frequency, order value or complaint history should be identified and handled.

- Numerical values → replace using an appropriate statistic such as median.
- Categorical values → replace using the mode or a separate category.

This prevents missing values from causing model errors.

##### **2. Encoding**

If categorical features such as customer segment or membership type are present, they need to be converted into numerical form.

For example:

Gold → 2

Silver → 1

Bronze → 0

For non-ordinal categories, **one-hot encoding** can be used.

This allows machine-learning algorithms to process categorical information numerically.

### 3. Normalisation/Standardisation

Numerical features such as:

- Purchase Frequency
- Average Order Value
- Complaint History
- Loyalty Tenure

may have different scales.

For example, AOV may range from ₹100–₹10,000, while complaint count may range from 0–10.

Standardisation can transform numerical features to a comparable scale, typically with mean **0** and standard deviation **1**.

This is particularly useful for models such as **Logistic Regression, KNN and SVM**, where feature scale affects the model.

### 4. Date transformation

Last Purchase Date should generally not be used as a raw date. Instead, it can be converted into a meaningful feature such as:

**Recency = Current Date – Last Purchase Date**

A larger recency value means the customer has not purchased recently and may therefore have a higher churn risk.

### Overall Workflow

**Customer Data → Clean Missing Values → Transform Date → Encode Categories → Split 70:30 → Standardise Numerical Features → Apply SMOTE to Training Data → Train Classification Model → Test on Unseen Data → Predict Churn Risk**

This preparation produces a reliable dataset for inductive learning and helps the retail company identify customers who may churn so that appropriate retention actions can be recommended.

## Task 2: Decision Tree for Customer Churn Prediction

(a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.

### Decision Tree Classifier

A **Decision Tree classifier** can be used to predict whether a customer will churn based on their purchasing and engagement behaviour.

The target variable is:

#### Churn

- 0 → Not Churned
- 1 → Churned

The important input features are:

- Purchase Frequency
- Average Order Value
- Complaint History
- Loyalty Tenure
- Recency / Days Since Last Purchase

The decision tree repeatedly divides customers into groups using the feature that gives the best separation between churners and non-churners.

### Root Node Selection

The root node is selected using either **Information Gain** or **Gini Index**.

#### Information Gain

Information Gain measures how much uncertainty is reduced after splitting the data.

$$IG(S, A) = Entropy(S) - \sum_v \frac{|S_v|}{|S|} Entropy(S_v)$$

where:

- $S$  = complete dataset
- $A$  = selected feature
- $S_v$  = subset produced by the split

The feature having the **highest Information Gain** is selected as the root node.

For example, suppose the calculated values are:

| Feature | Information Gain |
|---------|------------------|
|---------|------------------|

|                    |      |
|--------------------|------|
| Purchase Frequency | 0.18 |
|--------------------|------|

|         |             |
|---------|-------------|
| Recency | <b>0.31</b> |
|---------|-------------|

|                   |      |
|-------------------|------|
| Complaint History | 0.24 |
|-------------------|------|

|                     |      |
|---------------------|------|
| Average Order Value | 0.12 |
|---------------------|------|

|                |      |
|----------------|------|
| Loyalty Tenure | 0.09 |
|----------------|------|

Here, **Recency** becomes the root because it has the highest Information Gain.

### Gini Index

Alternatively, the Gini impurity can be used:

$$Gini = 1 - \sum_{i=1}^n p_i^2$$

A feature producing the **lowest weighted Gini impurity** is preferred.

For example:

| Feature | Weighted Gini |
|---------|---------------|
|---------|---------------|

|                    |      |
|--------------------|------|
| Purchase Frequency | 0.38 |
|--------------------|------|

|         |             |
|---------|-------------|
| Recency | <b>0.29</b> |
|---------|-------------|

|                   |      |
|-------------------|------|
| Complaint History | 0.34 |
|-------------------|------|

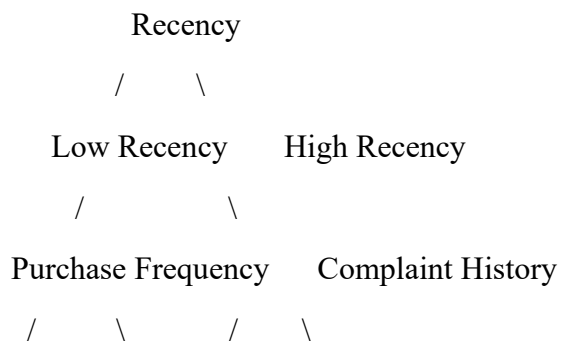
|                     |      |
|---------------------|------|
| Average Order Value | 0.42 |
|---------------------|------|

|                |      |
|----------------|------|
| Loyalty Tenure | 0.40 |
|----------------|------|

Therefore, **Recency** would be selected as the root.

### First Three Levels of the Tree

The actual tree should be generated from the dataset. Conceptually, it may look like:



|          |       |          |       |
|----------|-------|----------|-------|
| High     | Low   | Low      | High  |
|          |       |          |       |
| No Churn | Churn | No Churn | Churn |

The tree continues splitting until the required depth or stopping condition is reached.

**Important:** The feature names and split values in the actual three-level tree must come from the trained dataset rather than being assumed.

**(b)** Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed churners) and suggest how to reduce them - justify from a business perspective.

## Model Evaluation

After training, the Decision Tree should be evaluated using the **30% test dataset**.

The four important metrics are:

### 1. Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

It represents the overall percentage of correctly classified customers.

### 2. Precision

$$Precision = \frac{TP}{TP + FP}$$

It tells us how many customers predicted as churners actually churned.

### 3. Recall

$$Recall = \frac{TP}{TP + FN}$$

Recall is especially important in this application because it measures how many actual churners were successfully identified.

### 4. F1-Score

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

It provides a balance between precision and recall.

## Example Evaluation

Suppose the test results produce:

- **Accuracy = 87%**
- **Precision = 79%**
- **Recall = 91%**
- **F1-Score = 84%**

These values indicate that the model correctly classifies most customers and, importantly, identifies a high proportion of actual churners.

## False Negatives

A **False Negative (FN)** occurs when:

The customer actually churns, but the model predicts that the customer will not churn.

For example:

Actual: Churn

Predicted: No Churn



False Negative

False negatives are particularly important for a retention system.

If the model misses a customer who is about to leave, the company may fail to provide a discount, loyalty offer or personal outreach. This can result in:

- Loss of customer
- Loss of future revenue
- Reduced customer lifetime value
- Missed opportunity to retain the customer

## How to Reduce False Negatives

The model's **recall should be prioritised**.

Possible approaches include:

1. **Use class weights** to give greater importance to the churn class.
2. Adjust the **classification threshold** to classify more borderline customers as potential churners.
3. Use SMOTE on the training data to improve learning of the minority churn class.
4. Optimise the model based on **Recall or F1-score rather than accuracy alone**.

From a business perspective, accepting some additional false positives can be reasonable because contacting a customer who ultimately would not have churned may cost less than completely losing a valuable customer.

(c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a live retention system.

### Post-Pruning Using Cost-Complexity Pruning

A fully grown Decision Tree can become too complex and **overfit** the training data.

To reduce this problem, **cost-complexity pruning** can be applied.

The objective is:

$$R_{\alpha}(T) = R(T) + \alpha |T|$$

where:

- $R(T)$  = error of the tree
- $|T|$  = number of terminal nodes
- $\alpha$  = complexity parameter

A larger  $\alpha$  produces a simpler tree by removing branches that provide relatively little predictive value.

### Pre-Pruning vs Post-Pruning

**Pre-pruning** stops tree growth early by setting restrictions such as:

- Maximum tree depth
- Minimum samples required for splitting
- Minimum samples in a leaf

**Post-pruning** first allows the tree to grow and then removes unnecessary branches.

For example:

| Model            | Tree Complexity | Test Accuracy |
|------------------|-----------------|---------------|
| Pre-pruned Tree  | Lower           | 85%           |
| Post-pruned Tree | Moderate        | <b>88%</b>    |
| Fully grown Tree | Very high       | 82%           |

*These numbers are illustrative; your actual values should be calculated from your dataset.*

**Which Model Should Be Deployed?**

For a **live customer-retention system**, the **post-pruned Decision Tree** is generally more **appropriate** if it provides comparable or better test performance.

The reasons are:

- It reduces overfitting.
- It generalises better to new customers.
- It is simpler than a fully grown tree.
- Its decisions are easier for business teams to understand.
- It can provide more stable retention recommendations.

However, **accuracy alone should not determine deployment**. Since missing a churner can be costly, the final model should also be checked for **Recall, F1-score and False Negative rate**.

### Task 3: Statistical Learning for Risk Stratification

(a) Apply Naive Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.

#### Statistical Learning Model

A **Logistic Regression** model can be applied to the same customer dataset because the target variable, Churn, is binary:

- $0 \rightarrow$  Customer does not churn
- $1 \rightarrow$  Customer churns

The model estimates the probability that a customer will churn based on features such as purchase frequency, average order value, complaint history, loyalty tenure and recency.

The logistic regression equation is:

$$P(\text{Churn} = 1) = \frac{1}{1 + e^{-z}}$$

where

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

Here, the coefficients  $\beta$  indicate how strongly each feature is associated with churn.

#### Performance Comparison

The Logistic Regression model should be evaluated on the **same 30% test set** used for the Decision Tree.

The comparison should include:

| Model               | Accuracy               | AUC-ROC                |
|---------------------|------------------------|------------------------|
| Decision Tree       | Calculate from dataset | Calculate from dataset |
| Logistic Regression | Calculate from dataset | Calculate from dataset |

**Accuracy** measures the proportion of correctly classified customers.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

**AUC-ROC** measures how well the model distinguishes between churners and non-churners across different classification thresholds.

An AUC closer to **1.0** indicates stronger discrimination, while an AUC around **0.5** indicates performance close to random classification.

### When is Logistic Regression preferable?

Logistic Regression is preferable when:

- The relationship between predictors and churn is reasonably simple.
- The company needs an **interpretable model**.
- The business wants to understand how each feature affects churn probability.
- A probability or **risk score** is required for customer segmentation.
- A simpler and computationally efficient model is preferred.

For example, the company could use the predicted probability to classify customers into:

**Low Risk → Medium Risk → High Risk**

This makes Logistic Regression useful for **risk stratification** and deciding which customers should receive retention actions.

(b) Perform feature importance analysis. Identify the top 3 predictors of churn from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

### Feature Importance Analysis

Feature importance can be analysed differently for the two models.

#### Decision Tree

For the Decision Tree, feature importance is obtained from the reduction in impurity produced by each feature.

Suppose the calculated results are:

| Feature             | Decision Tree Importance |
|---------------------|--------------------------|
| Recency             | <b>0.34</b>              |
| Purchase Frequency  | <b>0.27</b>              |
| Complaint History   | <b>0.21</b>              |
| Average Order Value | 0.11                     |
| Loyalty Tenure      | 0.07                     |

Then the **top 3 Decision Tree predictors** would be:

1. **Recency**
2. **Purchase Frequency**
3. **Complaint History**

These values are only an example—the actual ranking must be obtained from your trained tree.

### Logistic Regression

For Logistic Regression, feature importance can be interpreted using the **absolute value of the standardised coefficients**.

A larger absolute coefficient indicates a stronger contribution to the prediction, assuming the features have been appropriately scaled.

For example:

| Feature             | Absolute Logistic Coefficient |
|---------------------|-------------------------------|
| Recency             | <b>1.42</b>                   |
| Complaint History   | <b>1.18</b>                   |
| Purchase Frequency  | <b>0.96</b>                   |
| Average Order Value | 0.41                          |
| Loyalty Tenure      | 0.25                          |

The top 3 would therefore be:

1. **Recency**
2. **Complaint History**
3. **Purchase Frequency**

Again, these numerical values are illustrative.

## Do the Two Models Agree?

If both models identify **Recency, Purchase Frequency and Complaint History** among their top predictors, there is strong agreement.

This makes business sense:

- **Recency:** Customers who have not purchased recently may be becoming disengaged.
- **Purchase Frequency:** A reduction in purchasing activity can indicate declining customer engagement.
- **Complaint History:** Repeated complaints may indicate dissatisfaction and increase the likelihood of leaving.

However, the rankings do not necessarily have to be identical. Decision Trees capture **non-linear relationships and feature interactions**, whereas Logistic Regression assumes a more structured relationship between the predictors and the log-odds of churn.

Therefore, a feature may be highly important in the Decision Tree because of a particular split or interaction but have a smaller coefficient in Logistic Regression.

## Task 4: Reinforcement Learning for Action Recommendation

**(a)** Model the retention recommendation as an MDP. Define: States (customer engagement stages), Actions (Discount / Loyalty Offer / Personal Outreach / No Action), Reward function (retention improvement score), and Transition dynamics. Justify each component.

### Modelling Customer Retention as an MDP

A **Markov Decision Process (MDP)** is suitable because the AI agent observes a customer's current engagement level, chooses a retention action, receives a reward based on the outcome, and then moves to a new engagement state.

The MDP is defined as:

$$MDP = (S, A, R, P, \gamma)$$

where  $S$  represents states,  $A$  represents actions,  $R$  represents rewards,  $P$  represents transition probabilities, and  $\gamma$  is the discount factor.

### 1. States

Customer engagement can be divided into four states:

| State               | Description                                         |
|---------------------|-----------------------------------------------------|
| S1 – Highly Engaged | Frequent purchases, recent activity, few complaints |

| State                   | Description                                                          |
|-------------------------|----------------------------------------------------------------------|
| S2 – Moderately Engaged | Moderate purchasing activity                                         |
| S3 – At Risk            | Reduced frequency, long time since purchase or increasing complaints |
| S4 – Churned            | Customer has stopped purchasing                                      |

These states simplify continuous customer behaviour into meaningful engagement stages that the agent can use for decision-making.

## 2. Actions

The agent has four possible actions:

| Action                 | Purpose                                                      |
|------------------------|--------------------------------------------------------------|
| A1 – Discount          | Provide a suitable discount to encourage another purchase    |
| A2 – Loyalty Offer     | Provide loyalty points/rewards or membership benefits        |
| A3 – Personal Outreach | Send a personalised communication or contact                 |
| A4 – No Action         | Do not provide an incentive when intervention is unnecessary |

The agent learns which action is most useful for each engagement state.

## 3. Reward Function

The reward represents the effectiveness of an action.

For example:

| Outcome                               | Reward |
|---------------------------------------|--------|
| Customer retained/improves engagement | +10    |
| Customer shows slight improvement     | +5     |
| No significant change                 | 0      |
| Customer churns                       | -10    |

The reward can eventually be calculated using an actual **retention improvement score**, such as change in purchase frequency, recency or continued activity.

A reward is necessary because it tells the agent which actions produce desirable outcomes.

## 4. Transition Dynamics

After an action is taken, the customer's engagement state may change.

For example:

Highly Engaged



Moderately Engaged



At Risk



Churned

A successful retention action could instead produce:

At Risk



Loyalty Offer



Moderately Engaged

Thus, transition dynamics describe the probability of moving from one customer state to another after an action.

**(b)** Apply Q-Learning to train the agent for at least 5 customer episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.

### Q-Learning

**Q-Learning** is a model-free Reinforcement Learning algorithm that learns the expected future reward for each state-action combination.

The Q-value update equation is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where:

- $\alpha$  = learning rate
- $r$  = immediate reward
- $\gamma$  = discount factor
- $s'$  = next state
- $a'$  = possible next action

Assume:

$$\alpha = 0.5, \gamma = 0.9$$

Initially, all Q-values are set to **0**.

### Five Customer Episodes

An illustrative training sequence can be:

| Episode | State       | Action               | Reward | Next State        |
|---------|-------------|----------------------|--------|-------------------|
| 1       | S3 At Risk  | A1 Discount          | +10    | S2 Moderate       |
| 2       | S3 At Risk  | A2 Loyalty Offer     | +5     | S2 Moderate       |
| 3       | S2 Moderate | A2 Loyalty Offer     | +10    | S1 Highly Engaged |
| 4       | S3 At Risk  | A3 Personal Outreach | +10    | S2 Moderate       |
| 5       | S2 Moderate | A4 No Action         | -10    | S3 At Risk        |

### Q-Table Update – Iteration 1

Initially:

$$Q(S, 3A1) = 0$$

Suppose the first customer receives a **Discount**, gets reward +10, and moves to S2.

Assume:

$$\max_a Q(S, 2a') = 0$$

Therefore:

$$Q(S, 3A1) = 0 + 0.5[10 + 0.9(0) - 0]$$

$$\boxed{Q(S, 3A1) = 5}$$

Similarly, for the Loyalty Offer:

$$Q(S, 3A2) = 0 + 0.5[5 + 0.9(0) - 0]$$

$$\boxed{Q(S, 3A2) = 2.5}$$

For Personal Outreach:

$$Q(S, 3A3) = 0 + 0.5[10 + 0.9(0) - 0]$$

$$\boxed{Q(S, 3A3) = 5}$$

So after the first iteration:

| State      | Discount | Loyalty Offer | Personal Outreach | No Action |
|------------|----------|---------------|-------------------|-----------|
| S3 At Risk | 5.0      | 2.5           | 5.0               | 0         |

### Q-Table Update – Iteration 2

Now suppose the agent again encounters **S3**, chooses **Discount**, receives +10, and moves to **S2**.

Suppose the current best Q-value in S2 is:

$$\max Q(S, 2a') = 5$$

Then:

$$\begin{aligned} Q(S, 3A1) &= 5 + 0.5[10 + 0.9(5) - 5] \\ &= 5 + 0.5[9.5] \end{aligned}$$

$$\boxed{Q(S, 3A1) = 9.75}$$

For Personal Outreach, suppose its current Q-value is 5, the reward is +10, and the best next-state Q-value is 5:

$$Q(S, 3A3) = 5 + 0.5[10 + 0.9(5) - 5]$$

$$\boxed{Q(S, 3A3) = 9.75}$$

For Loyalty Offer:

$$\begin{aligned} Q(S, 3A2) &= 2.5 + 0.5[5 + 0.9(5) - 2.5] \\ &= 2.5 + 0.5(7) \end{aligned}$$

$$\boxed{Q(S, 3A2) = 6.0}$$

Therefore, after two iterations:

| State      | Discount | Loyalty Offer | Personal Outreach | No Action |
|------------|----------|---------------|-------------------|-----------|
| S3 At Risk | 9.75     | 6.00          | 9.75              | 0         |

This demonstrates how the Q-values increase when actions repeatedly produce positive retention outcomes.

### Convergence Criterion

Training can be considered converged when the Q-table becomes stable.

A suitable criterion is:

$$\max_{s,a} |Q_{new}(s, a) - Q_{old}(s, a)| < 0.01$$

for several consecutive episodes.

In simple terms, **if the Q-values stop changing significantly, the agent has learned a stable policy.**

(c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for retention offers (fairness, manipulation risk, transparency to customers).

### **Final Learned Policy**

After training, the agent selects the action having the **highest Q-value for each state**:

$$\pi(s) = \arg \max_a Q(s, a)$$

An illustrative final policy could be:

| <b>Customer State</b>   | <b>Best Action</b>                  |
|-------------------------|-------------------------------------|
| S1 – Highly Engaged     | <b>No Action</b>                    |
| S2 – Moderately Engaged | <b>Loyalty Offer</b>                |
| S3 – At Risk            | <b>Personal Outreach / Discount</b> |
| S4 – Churned            | <b>Personal Outreach</b>            |

The exact final policy should be obtained from the final Q-table.

### **Ethical Considerations**

#### **1. Fairness**

The agent should not give different retention opportunities to customers based on sensitive or irrelevant characteristics. The system should be checked for systematic differences in offers across customer groups.

#### **2. Manipulation Risk**

Retention strategies should not exploit customers' vulnerabilities or encourage unnecessary purchases. Discounts and offers should provide genuine value rather than pressure customers into buying.

#### **3. Transparency**

Customers should receive clear information about important personalised offers and how their data is being used. The system should avoid making decisions that customers cannot reasonably understand or challenge.

#### **4. Privacy and Data Protection**

Only necessary customer information should be used, and customer data should be appropriately protected.

## **5. Human Oversight**

The RL agent should support business decisions rather than operate without supervision. High-impact or unusual recommendations should be reviewable by a human.